

Doc. no. P0776R1
Date: 2017-11-10
Project: Programming Language C++
Audience: LWG
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Rebase the Parallelism TS onto the C++17 Standard

Table of Contents

1. [Revision History](#)
 - [Revision 0](#)
 - [Revision 1](#)
2. [1 Introduction](#)
3. [2 Stating the problem](#)
 - [Consistency of propagation traits](#)
4. [3 Propose Solution](#)
5. [4 Other Directions](#)
6. [5 Formal Wording](#)
7. [6 Acknowledgements](#)
8. [7 References](#)

Revision History

Revision 0

Original version of the paper for the 2017 pre-Albuquerque mailing.

Revision 1

Updated for Albuquerque to reflect revised TS Draft in the pre-meeting mailing. Additional fixes made following LWG review:

- Fixed order of keywords `inline` and `namespace`
- Do not try to open nested exception namespace as part of the `inline parallelism_v2` namespace
- Do not strike out parent clauses for subclauses (partially) restored in the latest draft
- Add `override` keyword to exception's `what` method
- Add missing includes of non-experimental headers
- Rename `experimental/execution_policy` header to match C++17 name
- Update feature macro values to reflect change of namespace
- Replace clause 3 with ISO standard wording when no new terms or definitions are introduced
- Define behavior when an exception is thrown from a parallel algorithm using new policies
- Restore policy objects for the two new policies

1 Introduction

The current working draft for the Parallelism TS is based on C++14. Many of the features were significantly revised and adopted into the C++17 standard, so this paper proposed revising the draft to reflect the new standard.

2 Stating the problem

The first Parallelism TS achieved its purpose, and its contents were adopted into C++17 by applying the paper [P0024R2](#) out of the Jacksonville 2016 meeting. However, there were several issues with the original TS that were resolved in the C++17 process, incorporating a further 11 papers and 4 LWG issue resolutions that were not applied back to the TS. Future TSes should be based on that more recent specification.

2.1 The Wrong Namespace Formulation is Used

There are two namespace conventions for use inside the `experimental` namespace intended for TSes. The simplest is to use an inline namespace nested directly inside `namespace std::experimental` named after the TS, and incorporating a version suffix. This is used when the intent is to land the TS features directly into `namespace std`, should they be incorporated into a future standard. The alternative is to have a named (non-inline) namespace nested inside `namespace std::experimental`, and an inline version-number namespace inside that. This form is intended for use when it is expected that the named namespace will wrap the TS facilities in `namespace std` in the event that they are incorporated into a future standard.

The parallelism TS uses the second formulation, while document the intended adoption plan corresponds to the first formulation, and this is how it was applied by [P0024R2](#).

2.2 `exception_list` is Underspecified

There were a number of issues with the `exception_list` facility proposed by the original TS. One major concern was that there was no way for a user to populate an `exception_list` of their own, if they wanted to provide their own parallel algorithms following the standard interface. The intent was always to nail this down as part of adopting the specification into the standard, but doing so turned out to be more controversial than expected, and ultimately this form of error handling was deferred to a future set of parallel execution policies.

The most prominent concerns are discussed in paper [P0322R0](#) but there is still no clear sign of progress on using this error-reporting scheme, or even if the `exception_list` class should be immutable. There are important design issues to reconsider for its use in the new `task_block` facility, given that the class is no longer needed for the parallel algorithms. It may be possible to resolve the problematic design concerns if the context is a class intended only for use in the `task_block` interface.

2.3 Exception-handling Strategy Changed Completely

The parallel algorithms in C++17 use a different error reporting strategy, which is deferred to the parallel execution policy (and by default, simply calls `terminate`). It would seem unhelpful to retain the rejected error reporting behavior in the TS. If it were to be retained, it would be specified differently, by adopting new execution policies that have that behavior, which would have no impact on the specification of the algorithms themselves, so the duplication within the TS would be redundant.

2.4 Dynamic Execution Policy was not Adopted

There were concerns about adopting execution policies with runtime state (among other concerns) so the dynamic execution policy was deliberately dropped from the feature set adopted for C++17. Given the implementation concerns, and a deliberate poll to remove the feature as part of standardization, its ongoing presence in the TS is questionable.

2.5 Policy Types and Values Have New Names

The names for the remaining three policies went through several revisions, and do not match the policy names in the TS. Given the policies for the intended semantics have been adopted, we should not have duplicate policies with different names in the TS.

2.6 Many Algorithms Changed Specification

A variety of concerns came up with the specification for the algorithms, questioning the assumption that the serial specification was adequate for the parallel form of each algorithm. Several algorithms required explicit wording to get the complexity correct. One or two algorithms lost their parallel form, as it was unhelpful (due to mandatory serial stages) in the presence of the more general parallel algorithms added by the TS. Similarly, it was not clear what the correct restrictions to impose on algorithms taking input and output iterators would be in order to support parallelization (through some kind of scatter/gather) approach, and requiring support in all cases essentially serialized those algorithms, even where better approaches may be known to work. Hence, the iterator categories for many algorithms were changed, which required separate declarations for each algorithm, rather than the table with a list of names specified in this TS. Porting the revised specification for each algorithm into the TS is of dubious benefit, when we could just defer to the standard directly,

2.7 Forward Progress Guarantees

Two papers, [P0296R2](#) and [P0299R1](#), provide the vocabulary essential to provide forward progress guarantees, and then apply that vocabulary to the standard library. This wording is found only in C++17, and not back-ported to the TS, so we need to at least incorporate the C++17 specification by reference.

3 Propose Solution

The resolution proposed by this paper is to update the reference to the base C++ Standard to refer to ISO 14882:2017, rather than 2014, and then strike the text for all components that were adopted (in some form) into C++17.

Essentially two components remain, `exception_list` remains underspecified, with no progress on understanding the needs or implementation details. However, the new `task_block` component is specified to report errors using this class, so it must remain, unless `task_block` is respecified to use an error handling strategy more compatible with C++17.

This paper further recommends opening library issues to address the concerns raised by the ongoing presence of `exception_list` in this TS.

Finally, this paper proposes using the inline namespace `parallelism_v2` to contain the contents of this experimental TS, rather than an inline version namespace inside a nested `parallel` namespace, that is not intended for integration with a future C++ standard.

4 Other Directions

Another option would be to apply all of the C++17 papers to the TS. However, that would not be as simple as applying the edits to the standard, as many of those papers adjust words that are not present in the TS. It is also not entirely clear that there is a benefit in duplicating the whole specification in this way.

Another idea is that some of the changes for C++17 were the 'minimal' changes necessary to get a well-supported specification at this time, with the expectation that a number of issues would be revisited in time, with more generic (or specific) specification to follow, allowing more parallelism with better error-handling strategies. It may be worth retaining the original TS specification as a place to perform those ongoing experiments. This direction is rejected by this paper as holding over far too much of a pre-standard specification with a broad set of defects unrelated to that task. It would be preferred to add targeted new features as those proposals arise.

One last option is to entirely strike the specification for `exception_list` given the issues that arose trying to nail down a complete specification. However, it remains an essential part of the error-handling strategy for `task_block`, and this paper does not aim to review existing components that are not directly impacted by the adoption of C++17. While a review of this design in the light of C++17 is encouraged, this paper is not the place for that review.

5 Formal Wording

Strike the sections ~~deleted~~ in the table of contents below, entirely from the working paper:

1. Scope
2. Normative references
3. ~~Terms and definitions~~
4. General
 1. Namespaces and headers
 2. Feature-testing recommendations
5. Execution policies
 1. ~~In general~~
 2. Header `<experimental/execution_policy>` synopsis
 3. ~~Execution policy type trait~~
 4. ~~Sequential execution policy~~
 5. ~~Parallel execution policy~~
 6. ~~Parallel+Vector execution policy~~
 7. Unsequenced execution policy
 8. Vector execution policy
 9. ~~Dynamic execution policy~~
 1. ~~execution_policy_construct/assign~~
 2. ~~execution_policy_object access~~
 10. Execution policy objects
6. Parallel exceptions
 1. ~~Exception reporting behavior~~
 2. Header `<experimental/exception_list>` synopsis
7. Parallel algorithms
 1. ~~In general~~
 1. ~~Requirements on user-provided function objects~~
 2. ~~Effect of execution policies on algorithm execution~~
 3. Wavefront Application
 4. ~~ExecutionPolicy algorithm overloads~~
 2. ~~Definitions~~
 3. Non-Numeric Parallel Algorithms
 1. Header `<experimental/algorithm>` synopsis
 2. ~~For each~~
 3. No vec
 4. Ordered update class
 5. Ordered update function template
 4. ~~Numeric Parallel Algorithms~~
 1. ~~Header `<experimental/numeric>` synopsis~~
 2. ~~Reduce~~
 3. ~~Exclusive scan~~
 4. ~~Inclusive scan~~
 5. ~~Transform reduce~~
 6. ~~Transform exclusive scan~~
 7. ~~Transform inclusive scan~~
8. Task Block
 1. Header `<experimental/task_block>` synopsis
 2. Class `task_cancelled_exception`
 1. `task_cancelled_exception` member function `what`
 3. Class `task_block`
 1. `task_block` member function template `run`
 2. `task_block` member function `wait`
 4. Function template `define_task_block`
 5. Exception Handling

In addition, make the following edits to the clauses that remain. Original clause numbering is retained for clarity, although it is expected that the clauses will be renumbered applying the edits above.

2 Normative references [parallel.references]

1. The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
 - ISO/IEC 14882:2017—1, Programming Languages — C++
2. ISO/IEC 14882:2017— is herein called the C++ Standard. The library described in ISO/IEC 14882:2017— clauses 17-30 20-33 is herein called the C++ Standard Library. The C++ Standard Library components described in ISO/IEC 14882:2017— clauses 25, 26.7 and 20.7.2 28, 29.8 and 23.10.10 are herein called the C++ Standard Algorithms Library.
3. Unless otherwise specified, the whole of the C++ Standard's Library introduction (C++14 §17)(C++17 §20) is included into this Technical Specification by reference.

1) To be published. Section references are relative to N3937.

3 Terms and definitions [intro.defs]

1. No terms and definitions are listed in this document.
2. ISO and IEC maintain terminological databases for use in standardization at the following addresses:
 1. — IEC Electropedia: available at <http://www.electropedia.org/>
 2. — ISO Online browsing platform: available at <http://www.iso.org/obp>

4.1 Namespaces and headers [parallel.general.namespaces]

1. Since the extensions described in this Technical Specification are experimental and not part of the C++ Standard Library, they should not be declared directly within namespace std. Unless otherwise specified, all components described in this Technical Specification are declared in namespace std::experimental::parallel::v2 parallelism v2.

[Note: Once standardized, the components described by this Technical Specification are expected to be promoted to namespace std. — end note]

2. Unless otherwise specified, references to such entities described in this Technical Specification are assumed to be qualified with std::experimental::parallel::v2 parallelism v2, and references to entities described in the C++ Standard Library are assumed to be qualified with std::.
3. Extensions that are expected to eventually be added to an existing header <meow> are provided inside the <experimental/meow> header, which shall include the standard contents of <meow> as if by

```
#include <meow>
```

4.2 Feature-testing recommendations [parallel.general.features]

1. An implementation that provides support for this Technical Specification shall define the feature test macro(s) in Table 1.

Table 1 — Feature Test Macro(s)

Doc. No.	Title	Primary Section	Name	Value	Header
N4505	Working Draft, Technical Specification for C++ Extensions for Parallelism	7	__cpp_lib_experimental_parallel_algorithm	201505	<experimental/algorithm>
					<experimental/exception_list> <experimental/execution_policy> <experimental/numeric>
P0155R0	Task Block R5	8	__cpp_lib_experimental_parallel_task_block	201510 201711	<experimental/exception_list>
					<experimental/task_block>
P0076R4	Vector and Wavefront Policies	5.7	__cpp_lib_experimental_execution_vector_policy	201707 201711	<experimental/algorithm>
					<experimental/execution>

5.2 Header <experimental/execution_policy> synopsis [parallel.execpol.synopsis]

```
#include <execution>

namespace std {
  namespace std::experimental {
    inline namespace parallelism v2 {
      inline namespace v2 {
```

```

// 5.3, Execution policy type trait
template<class T> struct is_execution_policy;
template<class T> constexpr bool is_execution_policy_v = is_execution_policy<T>::value;

// 5.4, Sequential execution policy
class sequential_execution_policy;

// 5.5, Parallel execution policy
class parallel_execution_policy;

// 5.6, Parallel+Vector execution policy
class parallel_vector_execution_policy;

// 5.9, Dynamic execution policy
class execution_policy;

namespace execution {
    // 5.7, Unsequenced execution policy
    class unsequenced_policy;

    // 5.8, Vector execution policy
    class vector_policy;

    // 5.10, execution policy objects
    inline constexpr sequenced_policy seq{ unspecified };
    inline constexpr parallel_policy par{ unspecified };
}
}
}
}

```

5.7 Unsequenced execution policy [parallel.execpol.unseq]

```
class unsequenced_policy{ unspecified };
```

1. The class `unsequenced_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be vectorized, e.g., executed on a single thread using instructions that operate on multiple data items.
2. The invocations of element access functions in parallel algorithms invoked with an execution policy of type `unsequenced_policy` are permitted to execute in an unordered fashion in the calling thread, unsequenced with respect to one another within the calling thread. [Note: This means that multiple function object invocations may be interleaved on a single thread. — end note]
3. [Note: This overrides the usual guarantee from the C++ standard, Section 4.6 [intro.execution] that function executions do not interleave with one another. — end note]
4. During the execution of a parallel algorithm with the `experimental::execution::unsequenced_policy` policy, if the invocation of an element access function exits via an uncaught exception, `terminate()` shall be called.

5.8 Vector execution policy [parallel.execpol.vec]

```
class vector_policy{ unspecified };
```

1. The class `vector_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be vectorized. Additionally, such vectorization will result in an execution that respects the sequencing constraints of wavefront application ([parallel.alg.general.wavefront]). [Note: The implementation thus makes stronger guarantees than for `unsequenced_policy`, for example. — end note]
2. The invocations of element access functions in parallel algorithms invoked with an execution policy of type `vector_policy` are permitted to execute in an unordered fashion in the calling thread, unsequenced with respect to one another within the calling thread, subject to the sequencing constraints of wavefront application (7.1.3) for the last argument to `for loop` or `for loop strided`.
3. During the execution of a parallel algorithm with the `experimental::execution::vector_policy` policy, if the invocation of an element access function exits via an uncaught exception, `terminate()` shall be called.

5.10 Execution policy objects [parallel.execpol.objects]

```

constexpr sequential_execution_policy seq{};
constexpr parallel_execution_policy par{};
constexpr parallel_vector_execution_policy par_vec{};
inline constexpr execution::unsequenced_policy unseq{};
inline constexpr execution::vector_policy vec{};

```

1. The header `<experimental/execution_policy>` declares a global object associated with each type of execution policy defined by this Technical Specification.

6.2 Header `<experimental/exception_list>` synopsis [parallel.exceptions.synopsis]

```

namespace std {
namespace std::experimental {
inline namespace parallelism v2 {
inline namespace v2 {

class exception_list : public exception
{
public:
    typedef unspecified iterator;
    using iterator = unspecified;

    size_t size() const noexcept;
    iterator begin() const noexcept;
    iterator end() const noexcept;

    const char* what() const noexcept override;
};
}
}
}
}

```

1. The class `exception_list` owns a sequence of `exception_ptr` objects. The parallel algorithms may use the `exception_list` to communicate uncaught exceptions encountered during parallel execution to the caller of the algorithm.

7.3.1 Header `<experimental/algorithm>` synopsis [parallel.alg.ops.synopsis]

```

#include <algorithm>

namespace std {
namespace std::experimental {
inline namespace parallelism v2 {
inline namespace v2 {
    template<class ExecutionPolicy,
             class InputIterator, class Function>
    void for_each(ExecutionPolicy&& exec,
                 InputIterator first, InputIterator last,
                 Function f);
    template<class InputIterator, class Size, class Function>
    InputIterator for_each_n(InputIterator first, Size n,
                           Function f);
    template<class ExecutionPolicy,
             class InputIterator, class Size, class Function>
    InputIterator for_each_n(ExecutionPolicy&& exec,
                           InputIterator first, Size n,
                           Function f);

namespace execution {
// 7.3.3, No vec
template<class F>
    auto no_vec(F&& f) noexcept -> decltype(std::forward<F>(f)());

// 7.3.4, Ordered update class
template<class T>
    class ordered_update_t;

// 7.3.5, Ordered update function template
template<class T>
    ordered_update_t<T> ordered_update(T& ref) noexcept;
}
}
}
}

```

8.1 Header `<experimental/task_block>` synopsis [parallel.task_block.synopsis]

```

namespace std {
namespace std::experimental {
inline namespace parallelism v2 {
inline namespace v2 {
    class task_cancelled_exception;

    class task_block;

    template<class F>
        void define_task_block(F&& f);

    template<class F>
        void define_task_block_restore_thread(F&& f);
}
}
}
}

```

```
}  
}
```

8.2 Class `task_cancelled_exception` [`parallel.task_block.task_cancelled_exception`]

```
namespace std {  
    namespace std::experimental {  
        inline namespace parallelism_v2 {  
            inline namespace v2 {  
  
                class task_cancelled_exception : public exception  
                {  
                public:  
                    task_cancelled_exception() noexcept;  
                    virtual const char* what() const noexcept override;  
                };  
            }  
        }  
    }  
}
```

8.3 Class `task_block` [`parallel.task_block.class`]

```
namespace std {  
    namespace std::experimental {  
        inline namespace parallelism_v2 {  
            inline namespace v2 {  
  
                class task_block  
                {  
                private:  
                    ~task_block();  
  
                public:  
                    task_block(const task_block&) = delete;  
                    task_block& operator=(const task_block&) = delete;  
                    void operator&() const = delete;  
  
                    template<class F>  
                        void run(F&& f);  
  
                    void wait();  
                };  
            }  
        }  
    }  
}
```

6 Acknowledgements

Thanks to Jared Hoberock for the initial review of this paper. Thanks to Lisa Lippincot and others for finding and fixing the many spelling mistakes!

7 References

The following papers and issues were applied to the C++ Working Paper as part of C++17:

- [P0024R2](#) The Parallelism TS Should be Standardized, Jared Hoberock
- [P0296R2](#) Forward progress guarantees: Base definitions, Torvald Riegel
- [P0299R1](#) Forward progress guarantees for the Parallelism TS features, Torvald Riegel
- [P0336R1](#) Better Names for Parallel Execution Policies in C++17, Pablo Halpern
- [P0394R4](#) Hotel Parallelifornia: terminate() for Parallel Algorithms Exception Handling, JF Bastien, Bryce Adelstein Lelbach
- [P0452R1](#) Unifying <numeric> Parallel Algorithms, Bryce Adelstein Lelbach
- [P0467R2](#) Iterator Concerns for Parallel Algorithms, Alisdair Meredith
- [P0502R0](#) Throwing out of a parallel algorithm terminates—but how?, JF Bastien, Bryce Adelstein Lelbach, H. Carter Edwards
- [P0518R1](#) Allowing copies as arguments to function objects given to parallel algorithms, David S. Hollman, Jonathan Lifflander, Michael Wong, Detlef Vollmann
- [P0523R1](#) Wording for CH 10: Complexity of parallel algorithms, Detlef Vollmann
- [P0574R1](#) Algorithm Complexity Constraints and Parallel Overloads, Anthony Williams, Bryce Adelstein Lelbach
- [P0623R0](#) Final C++17 Parallel Algorithms Fixes, Bryce Adelstein Lelbach, Alisdair Meredith, Anthony Williams
- [LWG 2687](#) {inclusive,exclusive}_scan misspecified
- [LWG 2689](#) Parallel versions of `std::copy` and `std::move` shouldn't be in order
- [LWG 2718](#) Parallelism bug in `[algorithms.parallel.exec]p2`
- [LWG 2727](#) Parallel algorithms with `constexpr` specifier

The following papers informed the discussion of the papers and issues above:

- [P0322R0](#) `exception_list`, Bryce Adelstein Lelbach, Alisdair Meredith, Jared Hoberock
- [P0333R0](#) Improving Parallel Algorithm Exception Handling, Bryce Adelstein Lelbach
- [P0451R0](#) Future-Proofing Parallel Algorithms Exception Handling, Bryce Adelstein Lelbach, Alisdair Meredith
- [P0571R0](#) Type Requirements for `<numeric>` Algorithms, Bryce Adelstein Lelbach